Overview of Statistical Learning and Modeling – 36-600

Lab 3T: Data Manipulation with dplyr
Due Wednesday, September 10, 11:59pm

Trishla Nair

Data

We'll begin by importing some astronomical data from a repository I've made to collect all the data we'll use throughout the course. These data are stored in `.Rdata` format; such data are saved via `R`'s `save()` function and loaded via `R`'s `load()` function. One wrinkle here: the data are stored on the web, so we also have to apply the `url()` function.

```
load(url("https://github.com/FSKoerner/F25-36600-data/raw/refs/heads/main/Buzzard_DC1.Rdata"))
set.seed(101)
s  <- sample(nrow(df), 4000)
df <- df[s, -c(7:13)]
```

`df` is a data frame with 4000 rows and 7 columns; the first six are *predictor variables* and the seventh, `redshift`, is the *response variable*, in the context of supervised learning. Redshift is a directly observable proxy for the distance of a galaxy from the Earth. (After all, tape measures aren't going to help us here.) In a statistical learning exercise, we might try to determine a statistical model that optimally associates the predictor variables with the `redshift`.

Question 1

Apply the `dim()`, `nrow()`, `ncol()`, and `length()` functions to `df`, so as to build intuition about what these functions output. (*Note*: we are not using `dplyr` yet, just base `R` functions here.) Do you know why `length()` returns the value it does? Ask me or the TA if you do not. (But fill in an answer below regardless to reinforce the answer.)

```
print(dim(df))
```

```
## [1] 4000    7
```

```
print(nrow(df))
```

```
## [1] 4000
```

```
print(ncol(df))
```

```
## [1] 7
```

```
print(length(df))
```

```
## [1] 7
```

a df is like a list of lists where there are 7 lists of 4000 elements each.

Question 2

Display the names of each column of `df`.

```
colnames(df)
```

```
## [1] "u"        "g"        "r"        "i"        "z"        "y"        "redshift"
```

Time for a digression.

A *magnitude* is a logarithmic measure of brightness that is calibrated such as to be approximately zero for the brightest stars in the night sky (Sirius, Vega, etc.). Every five-unit *increase* in magnitude is a factor of 100 *decrease* in brightness. So a magnitude of 20 means the object is 100 million times fainter than a star like Vega.

Magnitudes are generally measured in particular bands. Imagine that you put a filter in front of a telescope that only lets photons of certain wavelengths pass through. You can then assess the brightness of an object at just those wavelengths. So `u` represents a magnitude determined at ultraviolet wavelengths, with `g`, `r`, and `i` representing green, red, and infrared. (The `z` and `y` are a bit further into the infrared. Those letters don't represent words.)

So the predictor data consists of six magnitudes spanning from the near-UV to the near-IR.

Question 3

Use the base `R summary()` function to get a textual summary of the data frame. Do you notice anything strange? (Some values you might not expect?)

```
summary(df)
```

```
##        u                g               r                i         
##  Min.   :19.55   Min.   :18.51   Min.   :18.00   Min.   :17.74  
##  1st Qu.:26.37   1st Qu.:25.86   1st Qu.:25.10   1st Qu.:24.55  
##  Median :27.34   Median :26.94   Median :26.29   Median :25.74  
##  Mean   :33.06   Mean   :26.87   Mean   :25.88   Mean   :25.27  
##  3rd Qu.:28.51   3rd Qu.:27.74   3rd Qu.:27.00   3rd Qu.:26.47  
##  Max.   :99.00   Max.   :99.00   Max.   :29.41   Max.   :27.00  
##        z                y            redshift        
##  Min.   :17.56   Min.   :17.34   Min.   :0.03713  
##  1st Qu.:24.21   1st Qu.:23.86   1st Qu.:0.49941  
##  Median :25.35   Median :25.04   Median :0.74346  
##  Mean   :24.98   Mean   :25.85   Mean   :0.82800  
##  3rd Qu.:26.12   3rd Qu.:25.89   3rd Qu.:1.08971  
##  Max.   :99.00   Max.   :99.00   Max.   :1.99975
```

```
The max for some columns are 99.
```

---

dplyr

Below, we will practice using `dplyr` package functions to select rows and/or columns of a data frame. `dplyr` is part of the `tidyverse`, and is rapidly becoming the most oft-used way of transforming data. To learn more about "transformatory" functions, read through today's class notes, then through Chapter 5 of the online version of *R for Data Science* (see the syllabus for the link, or just Google for it). In short, you can:

action	function
pick features by name	`select()`
pick observations by value	`filter()`
pick observations by category	`group_by()`
create new features	`mutate()`
reorder rows	`arrange()`
collapse rows to summaries	`summarize()`

A cool thing about `dplyr` is that you can use a piping operator (%>%) to have the output of one function be the input to the next. And you don't have to have only `dplyr` functions within the flow; for instance, you could pipe the first two rows of your data frame to the `head()` function (which just displays the first few rows of an object):

```
suppressMessages(library(tidyverse))
head(df[,1:2])                          # base R
```

```
##              u       g
## 2873   23.5907 21.6827
## 108639 27.1252 26.2652
## 19665  27.4013 26.7235
## 87391  27.9354 27.0586
## 35772  27.7733 27.4076
## 46326  27.1742 26.9353
```

```
df %>% select(., u, g) %>% head(.)      # dplyr
```

```
##              u       g
## 2873   23.5907 21.6827
## 108639 27.1252 26.2652
## 19665  27.4013 26.7235
## 87391  27.9354 27.0586
## 35772  27.7733 27.4076
## 46326  27.1742 26.9353
```

Note: if you've used `tidyverse` syntax before, you might notice that we don't always even need the `.`, which indicates where the previous object should be passed into the subsequent function. In fact, we can exclude it entirely, but then `R` will assume the preceding output should go into the succeeding function's **first** argument. If that's what you want, and you don't need to specify any other arguments, you can even ditch the parentheses entirely. E.g., the following line does exactly the same thing as the last two:

```
df %>% select(u, g) %>% head()              # dplyr, but shorter
```

```
##              u       g
## 2873   23.5907 21.6827
## 108639 27.1252 26.2652
## 19665  27.4013 26.7235
## 87391  27.9354 27.0586
## 35772  27.7733 27.4076
## 46326  27.1742 26.9353
```

Now let's do a few exercises. Ask us for extra background/detail! (Or, computing problems tend to often be solved in the real world, tap into, e.g., StackOverflow and *R for Data Science*.)

Question 4

Grab all data for which `i` is less than 25 and `g` is greater than 22, and output it in order of increasing `y`. (Remember: you combine conditions with & for "and" and | for "or".) Show only the first six lines of output. Note that `head()` by default shows the first six rows of the input data frame.

```
df %>%arrange(., y)%>%filter(., (i<25 & g>22)) %>% head(., 6)
```

```
##         u       g       r       i       z       y redshift
## 1 24.1858 22.0232 20.3907 19.5766 19.1923 18.9443 0.478264
## 2 24.3510 22.1041 20.4449 19.6847 19.2919 19.0380 0.445455
## 3 24.4236 22.5979 21.0056 19.8446 19.4433 19.0918 0.661301
## 4 24.8989 22.8297 21.2118 20.0455 19.6235 19.2732 0.644797
## 5 23.7571 22.1921 20.6919 19.9896 19.6521 19.3608 0.486188
## 6 24.1374 22.1437 20.5673 19.9751 19.6583 19.4766 0.378247
```

Question 5

To get a quick, textual idea of how the data are distributed: select the `g` column, pipe it to the `round()` function, then pipe the output of `round()` to `table()`. You should notice something strange about the output...the same thing you should have noticed when answering Question 3.

```
df%>%select(.,g)%>%round()%>%table()
```

```
## g
##   19   20   21   22   23   24   25   26   27   28   29   30   31   32   34   99
##    5    5   27   46   99  206  388  722 1224  946  247   51   13    9    1   11
```

Time for another digression. Domain scientists are not bound by the `R` convention of using `NA` when data are "not available," or missing. Sometimes the domain scientists will tell you what they use in place of `NA`, sometimes not. In those latter cases, one can usual infer the values. Astronomers for some reason love using

-99, -9, 99, etc., to represent missing values. The values of 99 in the table you generated in Question 5 actually represent missing data.

We could change the values of 99 to `NA`, but here we will just filter those rows with values of 99 out of the data frame.

Question 6

Use `filter()` to determine how many rows contain 99's. Since 99's can appear in different columns, you will need to combine conditions together with logical "and"s or "or"s. *Note*: only four of the six columns have 99's in them.

```
df%>%filter(., (u ==99 | g == 99 | z == 99 | y == 99))%>%head()
```

```
##         u       g       r       i       z       y redshift
## 1 99.0000 28.6780 27.9336 26.5562 25.0876 24.8142 1.724650
## 2 99.0000 26.9491 25.8114 24.8964 24.6316 24.5645 0.764673
## 3 99.0000 29.6675 27.7214 26.7440 26.5730 25.9962 0.382349
## 4 99.0000 28.0462 27.8069 26.9388 27.5762 27.8884 0.712806
## 5 28.2424 29.4176 28.1202 26.6837 26.2754 99.0000 0.862042
## 6 99.0000 28.9702 27.8574 26.1651 25.0758 24.5420 1.633220
```

Question 7

Now repeat Question 6 (in a sense), and remove all the rows that contain 99's, saving the new data frame as `df.new`. *Hint*: the opposite of, e.g., `u == 99 | g == 99` is `u != 99 & g != 99`, etc. If you apply `dim()` to the new data frame, you should see that 3607 rows are retained.

```
df.new <- df%>%filter(., (u !=99 & g != 99 & z != 99 & y != 99))
dim(df.new)
```

```
## [1] 3607    7
```

The data we are working with have no factor variables, so let's create one on the fly, using the `df.new` object you created in the last question (make sure to remove the chunk option `eval=FALSE`):

```
type    <- rep("FAINT", nrow(df.new))
w       <- which(df.new$i < 25)
type[w] <- "BRIGHT"
type    <- factor(type)
unique(type)
```

```
## [1] BRIGHT FAINT
## Levels: BRIGHT FAINT
```

```
df.new  <- cbind(type, df.new)
```

So, we've defined our factor variable using character strings, and then coerced the vector of strings into a factor variable with two levels. Note that by default, the levels are ordered alphabetically. For most factor variables, this wouldn't be, in any sense, a *natural* ordering. You can override that default behavior if there actually is a natural ordering to any given factor variable. See the documentation for `factor()` to see how to do that.

Question 8

Use `group_by()` and `summarize()` to determine the numbers of `BRIGHT` and `FAINT` galaxies in the dataset. (If you are adventuresome: try implementing the `tally()` function instead of `summarize()`. For our purposes here, it's actually easier.)

```
df.new%>%group_by(., type)%>%tally(.,)%>%as_tibble(.name_repair = "unique")
```

```
## # A tibble: 2 x 2
##   type       n
##   <fct>  <int>
## 1 BRIGHT  1295
## 2 FAINT   2312
```

Question 9

Repeat Question 8, but show the median value of the `u` magnitude instead of the counts in each factor group. (You do need `summarize()` here; `tally()` won't help you.)

```
df.new%>%group_by(., type)%>%summarize(Median = median(u))
```

```
## # A tibble: 2 x 2
##   type   Median
##   <fct>   <dbl>
## 1 BRIGHT   25.9
## 2 FAINT    27.7
```

Time for yet another digression.

Magnitudes of galaxies at particular wavelengths are heavily influenced by two factors: physics (what is going on with the gas, dust, and stars within the galaxy itself), and distance (the further away a galaxy is, the less bright it tends to be, so the magnitude generally goes up). To attempt to mitigate (somewhat, but not necessarily completely) the effect of distance, astronomers often use *colors*, which are defined as differences in magnitude for two adjacent filters.

Question 10

Use `mutate()` to define two new columns for `df.new`: `gr`, which would be the `g` magnitude minus the `r` magnitude, and `ri`, for `r` and `i`. Save your result as `df.newer`.

```
df.newer<- df.new%>%mutate(., gr = g-r)%>%mutate(., ri = r-i)
df.newer%>%head()
```

```
##     type       u       g       r       i       z       y redshift     gr     ri
## 1 BRIGHT 23.5907 21.6827 20.0759 19.4611 19.0885 18.8831 0.385261 1.6068 0.6148
## 2  FAINT 27.1252 26.2652 25.5893 25.0175 24.6461 24.0259 1.450010 0.6759 0.5718
## 3  FAINT 27.4013 26.7235 26.6074 26.5520 25.7460 25.4095 1.431700 0.1161 0.0554
## 4  FAINT 27.9354 27.0586 26.3402 25.6980 25.0166 24.2701 1.743960 0.7184 0.6422
## 5  FAINT 27.7733 27.4076 26.0681 25.3545 25.0698 24.9984 0.882537 1.3395 0.7136
## 6  FAINT 27.1742 26.9353 26.2515 26.2313 26.0436 25.3952 0.329310 0.6838 0.0202
```

Question 11

Are the mean values of `gr` and `ri` roughly the same for `BRIGHT` galaxies versus `FAINT` ones? I haven't the faintest. Use `dplyr` functions to attempt to answer this question.

```
means_gr <-df.newer%>%group_by(., type)%>%summarize(Mean = mean(gr))
means_ri <-df.newer%>%group_by(., type)%>%summarize(Mean = mean(ri))
print(means_gr)
```

```
## # A tibble: 2 x 2
##   type    Mean
##   <fct>  <dbl>
## 1 BRIGHT 0.942
## 2 FAINT  0.694
```

```
print(means_ri)
```

```
## # A tibble: 2 x 2
##   type    Mean
##   <fct>  <dbl>
## 1 BRIGHT 0.620
## 2 FAINT  0.558
```

Question 12

Let's go back to hypothesis testing for a moment. Let's extract two vectors of data from `df.newer` (again, remove the chunk option `eval=FALSE`):

```
gr.faint  <- df.newer$gr[df.newer$type == "FAINT"]
gr.bright <- df.newer$gr[df.newer$type == "BRIGHT"]
length(gr.faint)
```

```
## [1] 2312
```

```
length(gr.bright)
```

```
## [1] 1295
```

The first vector has all the `gr` colors for faint galaxies, and the second vector has all the ones for bright galaxies. The code here *should* be familiar given what we covered in Week 1, but if you are not sure what this code is doing, call me or the TA over!

Here are two histograms, one for `gr.faint` and one for `gr.bright` (again, remove the chunk option `eval=FALSE`):

```
hist(gr.faint, prob = TRUE, main = NULL, col = alpha("firebrick", 0.4),
     ylim = c(0, 1.2), xlab = "color", breaks = seq(-1.5, 5.5, by = 0.2))
hist(gr.bright, prob = TRUE, main = NULL, col = alpha("dodgerblue", 0.4),
     breaks = seq(-1.5, 5.5, by = 0.2), add = TRUE)
```

Now: test the hypothesis that the means of the distributions that `gr.faint` and `gr.bright` are sampled from are the same. Use a two-sample t-test. What do you conclude? Is a two-sample t-test actually strictly appropriate here? How would you know? (Even if the answer is no, do the t test anyway.)

```
print(t.test(gr.faint, gr.bright))
```

```
##
##  Welch Two Sample t-test
##
## data:  gr.faint and gr.bright
## t = -14.389, df = 2979.8, p-value < 2.2e-16
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  -0.2816093 -0.2140635
## sample estimates:
## mean of x mean of y
## 0.6942116 0.9420480
```

Because the calculated p-value is extremely small, there is a significant difference between the means